

Full Length Article

DSTFGCN: A dynamic spatial-temporal fusion graph convolution network for traffic flow forecasting

Tianyi Pan^a, Xinyuan Zhou^a, Shiyong Lan^{a,*}, Wenwu Wang^b, Hongyu Yang^a, Zheng Li^a, Zhiang Hou^c, Yao Ren^a

^a College of Computer Science, Sichuan University, Chengdu, China

^b Department of Electrical and Electronic Engineering, University of Surrey, Guildford, UK

^c National Key Laboratory of Fundamental Science on Synthetic Vision, Sichuan University, Chengdu, China

ARTICLE INFO

Keywords:

Traffic forecasting
Dynamic graph generation
Spatial-temporal graph convolution

ABSTRACT

Traffic flow prediction is one of the core technology of Intelligent Transportation System. Its fundamental challenge is to effectively model the complex spatial-temporal dependencies. Although extensive research has been conducted in this field, the limitations of current methods restrict their effectiveness in accurate predictions. For temporal dependence, existing methods based on recurrent neural networks only focus on local dependencies and ignore global dependencies. For spatial dependencies, existing methods use predefined or adaptive adjacency matrices that cannot accurately reflect the relationships between real traffic flow. To overcome these limitations, we propose a dynamic spatial-temporal fusion graph convolution network (DSTFGCN). In the temporal aspect, we introduce gated dilated causal convolution to capture the local dependencies and node-independent temporal graph convolution to capture the global dependencies specific to each node. In the spatial aspect, we propose a dynamic graph convolution block. It can construct dynamic graphs based on the characteristics of the input data and aggregate both local and global spatial dependencies. Experiments on six real-world datasets have shown that DSTFGCN outperforms current mainstream methods. The codes are available at <https://github.com/SYlan2019/DSTFGCN>.

1. Introduction

Intelligent Transportation Systems (ITS) have become an integral component of modern smart cities (Fan et al., 2025). Traffic flow prediction is a key enabling technology for ITS, aiming to forecast future traffic conditions based on historical observations. Accurate traffic prediction is essential for improving the efficiency of road networks (Dong & Zhang, 2025).

Extensive efforts have been devoted to traffic flow prediction. Traditional statistical approaches, such as the autoregressive integrated moving average (ARIMA) model (Abellana, 2021) and support vector regression (SVR) (Wu et al., 2004), yet these methods rely on the assumption of data stationarity, which limits their capacity to capture the complex non-linear spatio-temporal correlations inherent in traffic conditions (Weng et al., 2025). In recent years, deep learning models, including convolutional neural networks (CNNs) (Lin et al., 2019) and recurrent neural networks (RNNs) (Li et al., 2021), have achieved substantial progress and have been widely adopted for traffic forecasting

(Chen et al., 2025). Nevertheless, a fundamental challenge remains: how to effectively model the spatio-temporal correlations underlying traffic dynamics.

To model temporal correlations, many existing approaches adopt RNNs (Li et al., 2021), including long short-term memory (LSTM) (Seo et al., 2016) and gated recurrent unit (GRU) (Yao et al., 2018). However, RNN-based models are often hindered by the vanishing-gradient issue, which makes learning long-range dependencies challenging (Luo et al., 2024). More recently, Transformers have achieved strong performance across a wide range of domains (Chen et al., 2023b). Accordingly, a number of Transformer-based methods have been proposed for traffic flow prediction and have reported competitive results (Jiang et al., 2023; Liu et al., 2023). These approaches typically employ temporal self-attention to capture dynamic temporal dependencies, but they can be computationally expensive. Moreover, traffic dynamics may vary substantially across nodes even when they are spatially close. As illustrated in Fig. 1, the traffic flow at road 1 changes rapidly, whereas that at road 2 remains relatively stable. This observation motivates modeling

* Corresponding author.

E-mail addresses: 2023223040073@stu.scu.edu.cn (T. Pan), 2024323040016@stu.scu.edu.cn (X. Zhou), lanshiyong@scu.edu.cn (S. Lan), w.wang@surrey.ac.uk (W. Wang), 363879736@qq.com (H. Yang), 1041897080@qq.com (Z. Li), 2023226045008@stu.scu.edu.cn (Z. Hou), 2023223045140@stu.scu.edu.cn (Y. Ren).

<https://doi.org/10.1016/j.neunet.2026.108989>

Received 4 October 2025; Received in revised form 30 March 2026; Accepted 12 April 2026

Available online 15 April 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

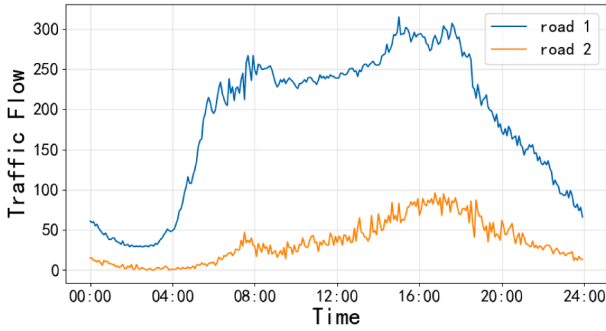


Fig. 1. Traffic characteristics of different nodes. The daily traffic of nodes NO.50 (node 1) and NO.92 (node 2) in the PESM08 dataset. The traffic flow of node 1 changes rapidly, while the traffic flow of node 2 is stable.

temporal dependencies in a node-aware manner to improve prediction accuracy. In standard Transformers, the same linear projections are commonly shared across nodes; such a unified mapping may be suboptimal for generating representations that emphasize node-specific patterns. In contrast, assigning node-specific parameters provides a more direct and flexible way to model heterogeneous, node-dependent traffic dynamics.

In terms of spatial modeling, early approaches often relied on CNNs (Lin et al., 2019), which are effective for learning patterns from Euclidean-structured data (Lou et al., 2024). However, traffic networks are inherently non-Euclidean, and CNN-based models may struggle to capture their complex spatial correlations (Huang et al., 2023). To address this limitation, graph convolutional networks (GCNs) (Ali et al., 2022) have been rapidly developed, and several studies (Bu et al., 2026; Yu et al., 2020) suggest that they can be more effective than grid-based convolutions for traffic forecasting. However, early GCN-based methods still face notable challenges. The adjacency matrix constructed from prior knowledge lacks task relevance and its expressiveness is inherently constrained by the quality of both the prior knowledge and the construction method. Weng et al. (2023).

More recently, adaptive adjacency matrices have been introduced to better model spatial dependencies, such as in Graph WaveNet (Wu et al., 2019), STFGCN (Ma et al., 2023) and AdaSTNet (Ta et al., 2022). These methods learn spatial relations via trainable node embeddings; however, the resulting adjacency matrix is usually fixed at inference time. As a consequence, their ability to adapt to input-dependent spatial variations in unseen test scenarios remains limited.

To address the limitations of traditional methods that ignore spatial dynamics and node-specific temporal dependencies, we propose the Dynamic Spatial-Temporal Fusion Graph Convolution Network (DSTFGCN) for traffic flow forecasting. DSTFGCN models both local and global temporal correlations. Local temporal correlations refer to short-range dependencies among adjacent time slots and are captured by gated dilated causal convolutions; the causal structure ensures only past information is used, effectively modeling short-term dynamics. Global temporal correlations denote long-range dependencies across the entire input horizon for each node. These node-specific global patterns are learned by a node-independent temporal graph convolution block. In this block each sensor is assigned a learnable projection matrix and each sensor's time slots are treated as vertices in a temporal graph, allowing graph convolution along the temporal dimension to aggregate node-specific global features. For spatial modeling, we employ diffusion graph convolution to capture local, topology-driven interactions among neighboring sensors. Here, local spatial dependencies are restricted to topologically adjacent nodes. We further introduce a dynamic graph convolution block whose graphs are generated from real-time inputs. By allowing edges between any pair of nodes (i.e., a fully connected input-dependent graph), this block captures global spatial dependencies, including cross-region interactions that may exist between any two sensors regardless of physical distance. We summarize our main contributions as follows:

- Local-global temporal modeling with node-specific global dependencies. We propose a temporal learning scheme that integrates gated causal convolutions for local, near-term temporal correlations with a node-independent temporal graph convolution for global temporal dependencies.
- Node-specific and periodicity-aware dynamic temporal graphs. We design a node-independent temporal graph convolution that learns node-specific global temporal patterns, and construct periodicity-driven temporal graphs where the graph slice is selected by the time slot within a period, enabling time-varying temporal dependency structures at different times of a period.
- Spatial correlation modeling via local-global fusion. We propose a spatial learning block that jointly captures (i) local topology-driven dependencies using diffusion graph convolution on the physical road network and (ii) global, input-conditioned semantic dependencies using a dynamic graph convolution whose adjacency is generated from the current traffic data, thereby fusing local and global spatial features in a unified framework.
- Experiments on six real road traffic datasets show an improvement in the performance of our proposed algorithm compared to several baselines.

2. Related work

2.1. Traffic forecasting

Traffic flow forecasting aims to predict future traffic conditions based on historical observations. Early studies primarily relied on traditional statistical approaches, such as ARIMA (Abellana, 2021) and historical average (HA) (Vlahogianni et al., 2004). However, these methods largely assume linear correlations and thus have limited capacity to model the complex nonlinear dynamics commonly observed in real-world traffic systems. Subsequently, machine learning-based methods, including KNN (Sun et al., 2017) and SVR (Wu et al., 2004), were introduced. Although these approaches can capture nonlinear relationships to some extent, they typically require handcrafted feature engineering, which limits scalability and robustness.

More recently, deep learning has driven substantial progress in traffic forecasting. For example, Yu et al. (2017) combined LSTM and CNN to predict traffic speed, and Ta et al. (2022) adopted graph neural networks (GNNs) for traffic flow prediction. Nevertheless, CNN-based models are mainly suited to grid-structured data, while LSTM-based models often struggle to capture long-range temporal dependencies. With the success of Transformers in natural language processing and computer vision, Transformer-based architectures have also been explored for traffic forecasting. PDFormer (Jiang et al., 2023) proposed a traffic delay-aware feature transformation module to address time-lag effects; and STAEformer (Liu et al., 2023) achieved state-of-the-art performance by leveraging spatio-temporal adaptive embeddings to capture intrinsic correlations in traffic time series. However, Transformer-based models are often computationally expensive.

2.2. Graph convolution network

Graph convolutional networks (GCNs) are effective for learning from graph-structured data and have been widely applied across diverse tasks (Luo et al., 2025). Broadly, GCN methods can be categorized into spectral- and spatial-domain approaches. Spectral-domain GCNs define convolutional filters in the graph Fourier space via the eigendecomposition of the graph Laplacian. However, computing the Laplacian eigenvalues (and eigenvectors) can be computationally expensive, which limits scalability. In contrast, spatial-domain GCNs perform convolution by aggregating information from local neighborhoods in the original graph domain. For example, GraphSAGE (Hamilton et al., 2017) learns node representations by sampling and aggregating features from a fixed-size

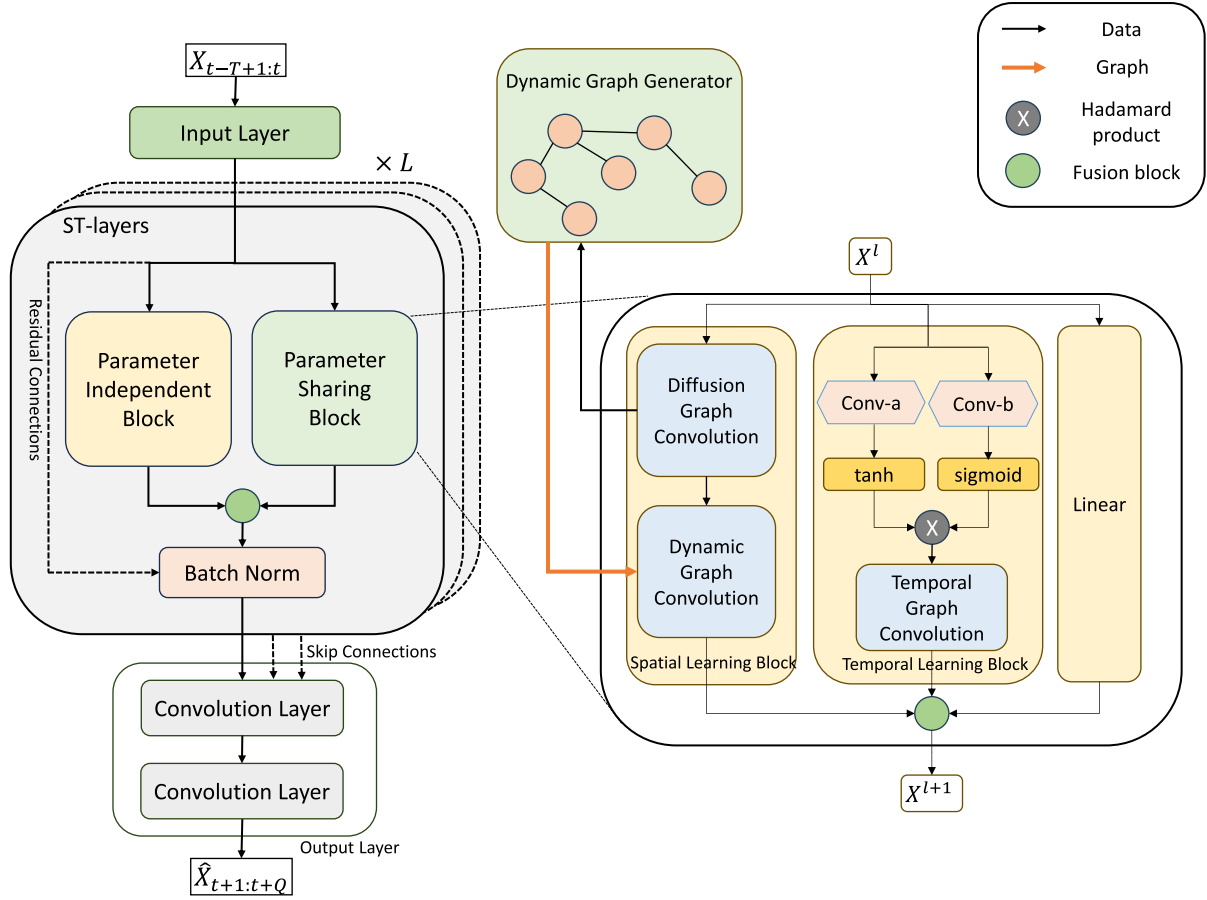


Fig. 2. Framework of DSTFGCN. It contains L identical layers. The input of the l th layer $X^l \in \mathbb{R}^{N \times T \times d}$. Each layer has two blocks, Parameter Independent Block and Parameter Sharing Block. Both of them contain a Temporal Learning Block, a Spatial Learning Block and a Linear Transformation Block. The details of the three blocks will be introduced in the next section. The difference is that the parameters of the Independent Block are different at each layer, while in Sharing Block are the same across layers.

set of neighbors, while GASTN (He et al., 2022) uses graph attention with global-local learning to capture near-far spatial dependencies for citywide mobile traffic forecasting.

Researchers have increasingly leveraged GNNs to model spatial correlations in traffic forecasting, with many efforts focusing on learning more informative graph structures to better characterize spatial dependencies. For example, DCRNN (Li et al., 2017) treats traffic flow as a diffusion process on a graph and employs bidirectional random walks to capture spatial correlations, while modeling temporal dependencies with a GRU. ASTGCN (Guo et al., 2019) combines attention mechanisms with graph convolution to model spatio-temporal dependencies. STS-GCN (Song et al., 2020) further improves forecasting by explicitly modeling localized spatio-temporal correlations.

To move beyond purely topology-based graphs, Graph WaveNet (Wu et al., 2019) introduces two learnable node-embedding matrices and derives an adaptive adjacency matrix by measuring similarities between source and target embeddings. Similarly, STFGCN (Ma et al., 2023) learns node embeddings to uncover latent spatial correlations. Other representative works include DSTAGNN (Lan et al., 2022), which proposes a dynamic spatio-temporal aware GNN. GDGCN (Xu et al., 2023) presents a generic dynamic graph convolutional network with parameter-sharing and parameter-independent blocks across stacked layers. AdpSTGCN (Zhang et al., 2024) employs multi-head attention to generate multi-view feature graphs, while DFAGCN (Liu et al., 2024) introduces a decomposition-with-feature-attention strategy to better capture local spatio-temporal patterns. More recently, COGCN (Guo et al., 2024) integrates multiple optimized road-network graphs and preserves

global-local feature consistency via contrastive learning, and STCL-AGA (Zhang et al., 2025a) incorporates spatio-temporal contrastive learning with adaptive graph augmentation and flow masking to better handle limited sensor data and evolving traffic patterns. ASTGCNs (Chen et al., 2023a) integrate adaptive graph convolution, attention, and TCN/ODE modules to jointly learn local-global spatio-temporal dependencies for traffic prediction and mitigate biases in explicit graph structures. In addition, DGCN-PTL (Chi et al., 2025) introduces collaborative pre-training for dynamic graph convolution, and HSTGNN (Wang et al., 2025) proposes a hybrid architecture with attention-based fusion. Weng et al. introduce SubGraph Learning (SGL) (Weng et al., 2025), which leverages learnable memory vectors to partition graphs into subgraphs and aggregates overlapping features, achieving significant GPU memory reduction while maintaining robust prediction performance. However, most of these methods still rely on either shared/adaptive graph structures, which may be insufficient to simultaneously capture input-dependent spatial dynamics and node-specific temporal heterogeneity in a unified and parameter-efficient manner.

3. Preliminaries

The notations used in this paper can be found in Table 1.

3.1. Road network

The road network is defined as an graph $G = (V, E, A)$. We consider each traffic sensor as a node in G , where V denotes the set of traffic

Table 1
Frequently used notations.

Notations	Definitions
X_t	Traffic flow in time slot t .
G	The road network graph.
V	The traffic sensors set.
N	The number of the traffic sensors.
E	The connectivity among traffic sensors.
A	The adjacency matrix of the road network G .
T	The number of input historical steps.
Q	The number of output predictable steps.
d	The hidden dimension.
N_t	The number of tensors constructed based on periodicity.
X^l	The input of the l th layer.
f	The traffic flow prediction function.

sensors and N is the number of traffic sensors. E is the set of edges. $A \in R^{N \times N}$ is the road adjacency matrix. If $E_{i,j} \in E$, the value of $A_{i,j}$ is 1, otherwise it is 0.

3.2. Feature matrix

Feature matrix $X \in R^{N \times T \times C}$ represents the traffic feature values of T time steps recorded on the road network, where N denotes the number of nodes and C denotes the number of features. In this work, we aim to research only traffic flow, hence $C = 1$.

3.3. Traffic flow forecasting

The goal of traffic flow prediction is to input the traffic flow in the past T time slots and to output the future Q time slots predicted flow. Consistent with most studies, we set T and Q to 12. The procedure to can be presented as:

$$\hat{X}_{t+1:t+Q} = f(X_{t-T+1:t}, G), \quad (1)$$

4. Methodology

4.1. Framework

The framework of is illustrated in Fig. 2. The traffic of historical T time slots $X \in R^{N \times T \times 1}$ is mapped to the hidden space $X \in R^{N \times T \times d}$ through the input layer. DSTFGCN consists of multiple identical layers. Inspired by Xu et al. (2023), each layer consists of two blocks: Parameter Independent Block and Parameter Sharing Block. The parameters of the Parameter Independent Block differ within different layers and are used to capture the layerwise patterns. The parameters of the Parameter Sharing Block are the same in all layers and are used to capture sharing patterns across layers. Each block consists of three parts: a Temporal Learning Block, a Spatial Learning Block, and a Linear Transformation Block, processing hidden states in temporal, spatial, and features respectively. The fusion block fuses the output and residuals through splicing and convolution operations to the output layer.

4.2. Temporal learning block

4.2.1. Local temporal correlation learning

We use a TCN (Temporal Convolutional Network) (Bai et al., 2018) layer that includes dilated causal convolution to capture the temporal dependence of nodes. Local temporal correlation should depend more on the recent past than on the distant past or future. The crucial convolution structure of TCN, causal convolution, allows it to see only past information when processing temporal correlation. Dilated convolution can increase the receptive field and obtain a rich representation of temporal. The structure of TCN makes it better at capturing local correlation. Compared with RNN based model, such as LSTM (Seo et al., 2016)

and GRU (Yao et al., 2018), TCN training can be completed in parallel, and gradient vanishing and exploding issues are not easy to occur. Given a sequence $x \in R^T$ and a convolution kernel $\theta \in R^M$, the convolution operation can be represented as follows:

$$\theta \star x(t) = \sum_{i=1}^M \theta(i)x(t - d_s \times i), \quad (2)$$

where $\star$ denotes the dilated causal convolution, t denotes the current time step, d_s denotes the dilation size and M denotes the size of the convolution kernel. In this work, we set $d_s = 1$ so that the convolution focuses on the most recent time steps, as we aim to capture local temporal dependencies dominated by near-term observations.

We use the TCN model proposed in Wu et al. (2019) to process local temporal correlation. Given the input $X \in R^{N \times T \times d}$, the calculation process of TCN takes the following form:

$$h = \tanh(\theta_1 \star X + b) \odot \text{sigmoid}(\theta_2 \star X + c), \quad (3)$$

where the $\theta_1, \theta_2 \in R^{d \times d'}$ and $b, c \in R^{d'}$ are trainable parameters, $\tanh()$ and $\text{sigmoid}()$ represent activation function, $\odot$ represents the Hadamard product and $\star$ denotes a dilated causal convolution. In terms of computational cost, the TCN module runs in $O(NTd^2)$ time per layer.

4.2.2. Global temporal correlation learning

Although dilated convolutions can enlarge the temporal receptive field, relating two distant time steps still requires multiple layers of information propagation. This gradual, long-range dependency modeling can make TCNs inefficient when handling long input sequences.

GDGCN (Xu et al., 2023) extends spatial relation learning to the temporal domain by introducing a temporal graph convolution block. Specifically, the T input time slots are treated as T vertices in a temporal graph, enabling global temporal correlations to be modeled flexibly from a graph perspective. However, the temporal adjacency matrix $A_{adp} \in R^{T \times T}$ is shared across all nodes, which forces different sensors to use the same temporal graph. As a result, node-specific temporal characteristics may not be adequately distinguished.

We adopt a node-independent temporal graph convolution block to model node-specific long-range temporal dependencies. Motivated by periodicity, we construct a set of N_t temporal graphs, represented as a 4th-order tensor $A_t \in R^{N_t \times N \times T \times T}$, where N_t denotes the number of graphs instantiated within one period. N_t is a hyperparameter and is typically set to 288 (one day) or 2016 (one week), depending on the sampling frequency and periodic characteristics of the dataset. For a given timestamp, we map it to a time slot t within the period and select the corresponding slice $A_{\phi(t)} \in R^{N \times T \times T}$ as the temporal graph used in computation. As the time slot t changes, the selected graph $A_{\phi(t)}$ also changes accordingly, meaning that different temporal dependency structures are activated at different times of the day/week; this enables the model to adapt to time-varying traffic dynamics. However, directly learning A_t can introduce an excessive number of parameters when N_t is large.

To reduce the parameter cost, we employ Tucker decomposition, a higher-order extension of PCA (Tucker, 1966), which factorizes a high-order tensor into a core tensor multiplied by factor matrices along each mode. Following Xu et al. (2023), we extend the original three-way factorization to our four-way temporal-graph tensor. As shown in Fig. 3, we parameterize A_t with five learnable components: a core tensor k and four factor matrices w_t, w_n, w_s, w_e . The temporal graph tensor $A_t \in R^{N_t \times N \times T \times T}$ is computed as follows:

$$A_t = \text{Softmax}(\text{ReLU}(k \times_1 w_t \times_2 w_n \times_3 w_s \times_4 w_e)), \quad (4)$$

where $\times_i$ denotes the tensor-matrix multiplication along the (i)th mode. $w_t \in R^{N_t \times H}$ encodes the N_t periodic temporal-graph instances, and $w_n \in R^{N \times L}$ encodes the N sensors. $w_s \in R^{T \times J}$ and $w_e \in R^{T \times K}$ are the factor matrices for the start and end time slots, respectively. The core tensor $k \in R^{H \times L \times J \times K}$ captures the interactions among latent components across the four modes. We apply ReLU to suppress negative values and then use Softmax to normalize the resulting adjacency, yielding

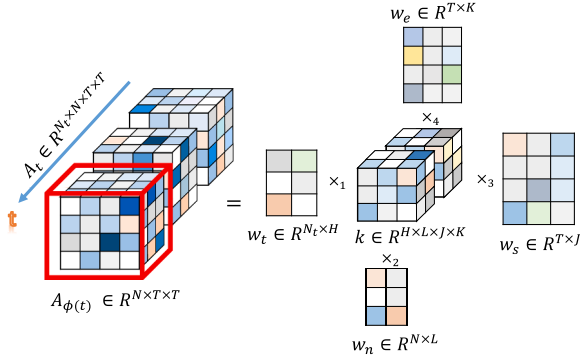


Fig. 3. Temporal graph construction. Each graph can be composed of a core tensor and four factor matrices.

a valid temporal graph. Algorithm 1 summarizes the temporal-graph generation procedure.

Algorithm 1 Temporal graph generation.

Input: Time slot: t

Output: Temporal adjacency tensor: $\mathbf{A}_t \in \mathbb{R}^{N_t \times N \times T \times T}$

- 1: Get embedding of time slot: $\mathbf{w}_t \in \mathbb{R}^{N_t \times H}$
 - 2: Get embedding of nodes: $\mathbf{w}_n \in \mathbb{R}^{N \times L}$
 - 3: Get embedding of start time slot: $\mathbf{w}_s \in \mathbb{R}^{T \times J}$
 - 4: Get embedding of end time slot: $\mathbf{w}_e \in \mathbb{R}^{T \times K}$
 - 5: Get core tensor: $\mathbf{k} \in \mathbb{R}^{H \times L \times J \times K}$
 - 6: $\mathbf{A}_t = \text{Softmax}(\text{ReLU}(\mathbf{k} \times_1 \mathbf{w}_t \times_2 \mathbf{w}_n \times_3 \mathbf{w}_s \times_4 \mathbf{w}_e))$
 - 7: **return** $\mathbf{A}_t$
-

4.2.3. Temporal graph convolution block

The temporal graph constructed in the previous section learned the temporal correlation. The purpose of the temporal graph convolution block is to aggregate global information from the temporal domain according to $A_{\phi(t)}$. Where $A_{\phi(t)} \in \mathbb{R}^{N \times T \times T}$ represents the impact between the T time slots of each node: the (i, j) -entry of $A_{\phi(t)} \in \mathbb{R}^{T \times T}$ reflects the effect of the i time slot of node n on the j time slot. The node-independent temporal graph convolution can be defined as:

$$Z_{tem} = \sum_{n=1}^N \sum_{k=1}^T A_{\phi(t),k}^n X_k^n, \quad (5)$$

where $A_{\phi(t),k}^n$ represents the slice of the temporal graph at the k th time slot at the n th node, X_k^n represents the slice of the input at the k th time slot of the n th node, $Z_{tem} \in \mathbb{R}^{N \times T \times d}$ represents the output of the temporal graph convolution block. The superscript n is the index of the node. The subscript k is the index of the time slot. The temporal graph convolution has time complexity $O(NT^2d)$ per forward pass.

4.3. Spatial learning block

Besides temporal correlation, there is also spatial correlation in traffic flow data. This work considers two spatial correlations: the correlation of physical topology and the correlation of node functions. The purpose of the spatial learning block is to learn these two spatial correlations.

4.3.1. Diffusion graph convolution block

The purpose of diffusion graph convolution block is to learn the correlation of physical topology. Inspired by Li et al. (2017), we treat traffic flow as diffusion signals on an adjacency matrix. Through the diffusion process, each node can aggregate information about its neighbors at a specific distance. Given input $X \in \mathbb{R}^{N \times T \times d}$ and $A \in \mathbb{R}^{N \times N}$ represents the original road topology. We get $\hat{A} = A + I$ for a road topology with a

self-loop, where I is the identity matrix. The process of diffusion graph convolution can be described as follows:

$$Z_{dif} = \sum_{k=0}^K (D_0^{-1} \hat{A})^k X, \quad (6)$$

where D_0 denotes the degree matrix of $\hat{A}$. The matrix $D_0^{-1} \hat{A} \in \mathbb{R}^{N \times N}$ can be viewed as a transition probability matrix, and the (i, j) -entry of $(D_0^{-1} \hat{A})^k$ denotes the probability that information diffuses from node i to node j after a k -order diffusion process. $Z_{dif} \in \mathbb{R}^{N \times T \times d}$ is the output of the diffusion graph convolution block. In terms of computational cost, the diffusion graph convolution scales approximately linearly with the size of the spatiotemporal tensor in traffic networks, requiring $O(NTd)$ per forward pass. After the input passes through the diffusion graph convolution block, it obtains an encoding of local spatial information. Next, global spatial information is learned through the dynamic graph convolution block.

4.3.2. Dynamic graph convolution block

After aggregating information from neighboring nodes, each node obtains a local spatial correlation. However, semantically correlated nodes may not be physically connected, and such global spatial correlations are input-dependent and dynamic. We therefore generate a dynamic graph to capture these evolving semantic relations. Given the input $X \in \mathbb{R}^{N \times T \times d}$, we project and aggregate to obtain a time-slot importance sequence $U \in \mathbb{R}^{N \times T}$:

$$U = \text{mean}(XW), \quad (7)$$

where $W \in \mathbb{R}^{d \times d_e}$ are trainable parameters, d_e denotes the embedding dimension. We use the mean operation to pool feature responses across the projected embedding dimension, providing a stable and compact importance score for each node-time slot pair in U while reducing noise and parameter sensitivity. Computing U requires a linear projection and mean pooling, and thus has time complexity $O(NTd)$ per forward pass. The (i, j) -entry of U is the aggregate weight of the j time slot of node i . The embedded representation of the node is obtained after aggregation:

$$S^n = \sum_{k=1}^T (Z_{dif,k}^n)^T U_k^n, \quad (8)$$

where $S^n \in \mathbb{R}^d$ is an embedded representation of node n , The superscript n denotes the index of node and T denotes the transpose. The subscript k is the index of the time slot. U_k^n reflects the relevance between the traffic pattern of a specific time slot of the k node and the overall regularities (S^n). U_k considered the differences in importance among different time slots and captured the dynamic behavior patterns of nodes in the temporal domain. The embedding of nodes can be used to characterize the information of traffic patterns in the road network.

In this work, we propose to evaluate the similarity of embedded representations between two nodes to reflect their traffic distribution. The more similar the embedding of two nodes, the more similar their traffic distribution. We utilize node embeddings to generate dynamic graphs as follows:

$$A_s = \text{Softmax}(\text{ReLU}(\frac{SS^T}{\sqrt{d}})), \quad (9)$$

where $S \in \mathbb{R}^{N \times d}$ is node embeddings. The (i, j) -entry of $A_s \in \mathbb{R}^{N \times N}$ reflects the degree of correlation between node i and node j . Since A_s is calculated based on the input, it changes with the input during the training and test phases, reflecting the dynamic changes in spatial correlation between nodes. In terms of computational cost, dynamic graph generation is dominated by the similarity computation SS^T , resulting in an overall time complexity of $O(N^2d)$ per forward pass. Algorithm 2 demonstrates the generation of dynamic graphs.

The dynamic graph convolution block aggregates global features based on the generated dynamic graph:

$$Z_{spa} = \sum_{k=1}^N A_{s,k} Z_{dif,k}, \quad (10)$$

Algorithm 2 Dynamic graph generation.**Input:**

$\mathbf{X} \in \mathbb{R}^{N \times T \times d}$: Traffic flow data
 d_e : Embedding dimension
 $\mathbf{Z}_{\text{dif}} \in \mathbb{R}^{N \times T \times d}$: Difference patterns

Output:

$\mathbf{A}_s \in \mathbb{R}^{N \times N}$: Dynamic adjacency matrix

- 1: **Step 1:** Compute embedding sequence
- 2: $\mathbf{U} = \text{mean}(\mathbf{X}\mathbf{W})$ $\triangleright \mathbf{W} \in \mathbb{R}^{d \times d_e}, \mathbf{U} \in \mathbb{R}^{N \times T}$
- 3: **Step 2:** Compute node embeddings
- 4: $\mathbf{S} = \sum_{k=1}^T (\mathbf{Z}_{\text{dif},k})^\top \mathbf{U}_k$ $\triangleright \mathbf{S} \in \mathbb{R}^{N \times d}$
- 5: **Step 3:** Generate dynamic graph
- 6: $\mathbf{A}_s = \text{Softmax}\left(\text{ReLU}\left(\frac{\mathbf{S}\mathbf{S}^\top}{\sqrt{d}}\right)\right)$
- 7: **return** $\mathbf{A}_s$

where $\mathbf{Z}_{\text{dif}}$ represents the output of the diffusion graph convolution block, it contains local characteristic information. $\mathbf{A}_s$ represents the dynamic graph generated, $\mathbf{A}_{s,k}$ represents the component of its k th node. The subscript k is the index of the node. $\mathbf{Z}_{\text{spa}} \in \mathbb{R}^{N \times T \times d}$ is the output of the dynamic graph convolution block, and it aggregates both local and global spatial features.

4.4. Linear block

The linear block aims to learn the transformation on the feature dimension:

$$\mathbf{Z}_{\text{lin}} = FC(\mathbf{X}), \quad (11)$$

where $\mathbf{X} \in \mathbb{R}^{N \times T \times d}$ is input. FC is implemented as a fully connected layer. $\mathbf{Z}_{\text{lin}} \in \mathbb{R}^{N \times T \times d}$ is the output of the linear block. The subscript lin represents linear. In terms of computational cost, the linear block is dominated by the fully-connected feature projection applied to each node and time step, yielding an overall time complexity of $O(NTd^2)$ per forward pass.

4.5. Parameter independent block

Parameter Sharing Block at different layers share the same set of parameters and are designed to learn common patterns across layers. The Parameter Independence Block utilizes different parameter sets at different layers and aims to learn independent patterns across layers. The internal structure of independent and shared block is similar, both contain temporal learning block, spatial learning block and linear block.

4.6. Loss function

Huber Loss (Huber, 1964) is a loss function that balances MSE and absolute error and provides robustness to outliers and differentiability. Therefore, Huber Loss has been widely used in the field of data mining. There are also some excellent works of traffic flow forecasting using huber loss (Lan et al., 2022; Li et al., 2022). Therefore, we choose huber loss as the loss function of the training process:

$$\mathcal{L} = \frac{1}{N \times Q} \sum_{n=1}^N \sum_{k=1}^Q h(\hat{x}_k^n, x_k^n), \quad (12)$$

$$h(\hat{x}, x) = \begin{cases} \frac{1}{2}(\hat{x} - x)^2, & |\hat{x} - x| \leq \delta \\ \delta|\hat{x} - x| - \frac{1}{2}\delta^2, & |\hat{x} - x| > \delta, \end{cases}$$

Where N and Q respectively represent the number of nodes and the time step, the subscript k represents the time slot, and the superscript n represents the node index, δ is the hyperparameter that balances the squared error. $\hat{x}$ denotes predicted value and x denotes the real value.

Table 2

Details of the datasets used in this work.

Dataset	Sensors	Time range	Time steps
PeMS03	358	09/2018–11/2018	26,208
PeMS04	307	01/2018–02/2018	16,992
PeMS07	883	05/2017–07/2017	28,224
PeMS08	170	07/2016–08/2016	17,856
METR-LA	207	03/2012–06/2012	34,272
PEMS-BAY	325	01/2017–05/2017	52,116

5. Experiments**5.1. Datasets**

We conducted experiments on six real-world traffic datasets: four traffic flow datasets (PeMS03, PeMS04, PeMS07, and PeMS08) and two traffic speed datasets (METR-LA and PEMS-BAY). The PeMS datasets are collected from the Caltrans Performance Measurement System (PeMS) (Chen et al., 2001). Raw traffic data was sampled at 5 min intervals. The input data were normalized by the Z-score normalization method. More details of the datasets are shown in Table 2.

5.2. Baselines

In the traffic flow forecasting task, we compared our DSTFGCN against 14 baseline models:

- Traditional time series method, including HA (Vlahogianni et al., 2004), ARIMA (Abellana, 2021), .
- Method based on graph neural network, including DCRNN (Li et al., 2017), Graph WaveNet (Wu et al., 2019), STSGCN (Song et al., 2020), DSTAGNN (Lan et al., 2022), GDGCN (Xu et al., 2023), DFAGCN (Liu et al., 2024), HSTGNN (Wang et al., 2025), LightST (Zhang et al., 2025b), DGCN-PTL (Chi et al., 2025).
- Transformer-based approach, including GMAN (Zheng et al., 2019), TFormer (Yan et al., 2021), ASTGNN (Guo et al., 2021).

In the traffic speed forecasting task, we compared our method against 6 baseline models. It includes two classic time-series methods (HA Vlahogianni et al., 2004 and ARIMA Abellana, 2021), three GNN-based methods (DCRNN Li et al., 2017, STGNN and DTAN Cao et al., 2025), and a Transformer-based method (STPFormer Chen et al., 2024).

5.3. Experimental settings

All of our experiments were conducted on NVIDIA GeForce RTX 3090 graphics cards. Consistent with most studies, we use the historical one-hour traffic flow to predict the traffic flow in the next hour, that is, the historical step T and the forecast step Q are set to 12. The PEMS datasets are split into training, validation, and test sets with a ratio of 6:2:2, while the traffic speed dataset is split with a ratio of 7:1:2. The batch size is set to 32 for PeMS08 and the traffic speed dataset, while it is set to 16 for the other datasets. The number of layers L is selected from 5,6,7,8. the embedding dimension d_e is set to 10. The learning rate is 1×10^{-3} and the weight decay coefficient is 1×10^{-4} . Other hyperparameters are detailed in the configuration file. All experiments were repeated 5 times and average results were reported.

5.4. Evaluation metrics

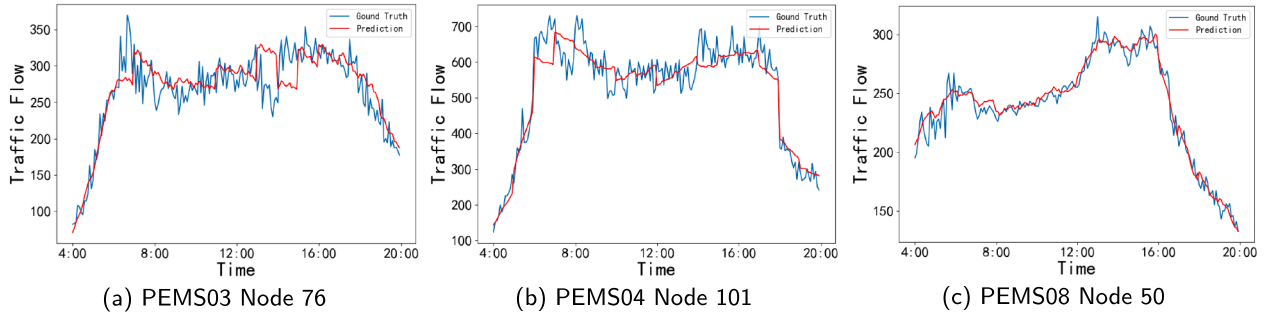
The mean absolute error (MAE), mean absolute percentage error (MAPE) and root mean square error (RMSE) were selected as evaluation metrics:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (13)$$

Table 3

Evaluations on PeMS datasets. The best results are in bold and the second best are underlined.

Dataset	PeMS03			PeMS04			PeMS07			PeMS08		
Metric	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
HA	31.58	52.39	33.78%	38.03	59.24	27.88%	45.12	65.64	24.51%	34.86	59.24	27.88%
ARIMA	35.41	47.59	33.78%	33.73	48.80	24.18%	38.17	59.27	19.46%	31.09	44.32	22.73%
DCRNN	18.18	30.31	18.91%	24.70	38.12	17.12%	25.30	38.58	11.66%	17.86	27.86	11.45%
Graph WaveNet	19.85	32.94	19.31%	25.45	39.70	17.29%	26.85	42.78	12.12%	19.13	31.05	12.68%
STSGCN	17.48	29.21	16.78%	21.19	33.65	13.90%	24.26	39.03	10.21%	17.13	26.80	10.96%
DSTAGNN	15.57	27.21	14.68%	19.30	31.46	12.70%	21.42	34.51	9.01%	15.67	24.77	9.94%
GDGCN	14.66	24.30	<u>13.94%</u>	<u>18.44</u>	<u>29.79</u>	12.52%	20.15	33.21	8.50%	14.82	23.87	<u>9.35%</u>
DFAGCN	15.19	24.16	14.99%	19.55	30.86	13.64%	22.14	35.30	9.55%	16.06	25.09	10.57%
HSTGNN	14.54	<u>24.17</u>	13.74%	18.57	29.88	12.58%	<u>19.85</u>	<u>33.04</u>	8.65%	14.78	23.84	9.62%
LightST	15.11	24.74	15.36%	18.92	31.31	12.70%	<u>20.78</u>	33.95	8.98%	15.43	24.52	9.84%
DGCN-PTL	<u>14.45</u>	24.74	14.64%	18.65	30.14	12.67%	20.13	33.17	<u>8.43%</u>	<u>14.59</u>	<u>23.41</u>	9.55%
GMAN	16.87	27.92	18.23%	19.14	31.60	13.19%	20.96	34.10	9.05%	15.31	24.92	10.13%
TFormer	15.03	25.32	15.36%	18.92	31.35	12.71%	20.75	34.06	8.97%	15.19	24.88	9.93%
ASTGNN	14.78	25.00	14.79%	18.60	30.91	<u>12.36%</u>	20.62	34.00	8.86%	15.00	24.70	9.50%
DSTFGCN	14.34	24.24	14.60%	18.18	29.75	12.18%	19.18	32.33	8.06%	13.45	23.07	8.80%

**Fig. 4.** Forecasting visualization of DSTFGCN on three datasets.**Table 4**

Average performance comparison on traffic speed datasets. The best results are in bold and the second best are underlined.

Method	METR-LA			PEMS-BAY		
	MAE	RMSE	MAPE	MAE	RMSE	MAPE
HA	5.19	10.44	13.02%	2.81	6.07	5.64%
ARIMA	5.68	6.97	13.89%	3.84	5.01	6.03%
DCRNN	3.60	7.59	10.50%	2.07	4.74	4.96%
STGNN	3.49	6.94	9.72%	1.83	4.20	4.29%
DTAN	<u>3.40</u>	6.74	9.85%	<u>1.72</u>	3.74	3.92%
STPFormer	3.41	<u>6.64</u>	<u>8.70%</u>	1.65	<u>3.83</u>	4.10%
DSTFGCN	3.19	6.50	9.30%	1.78	4.00	<u>3.97%</u>

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\%, \quad (14)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (15)$$

Where y_i and $\hat{y}_i$ represent the predicted and real traffic values, respectively.

5.5. Comparison results

Tables 3 and 4 present the overall results on six datasets (four traffic flow datasets and two traffic speed datasets). The experimental results represent the average performance over 12 prediction steps. Most of the baseline results are taken from officially published implementations (Liu et al., 2024). We observe that: (1) methods that model only temporal patterns (e.g., HA and ARIMA) perform poorly, indicating that traffic

data exhibit strong spatio-temporal dependencies beyond purely temporal correlations; (2) RNN-based approaches alone are often insufficient to capture complex temporal dynamics, especially for longer horizons; and (3) methods relying on a static graph structure tend to underperform, highlighting the importance of modeling the dynamic characteristics of spatial dependencies in real traffic systems.

Overall, DSTFGCN achieves consistently strong performance and outperforms existing baselines in most settings for traffic flow and traffic speed forecasting tasks. In particular, DSTFGCN attains the best MAE on most datasets, demonstrating its capability to effectively exploit spatio-temporal correlations in traffic data. Although DSTFGCN does not achieve the best RMSE and MAPE on PeMS03, this may be due to the larger traffic fluctuations in this dataset: optimizing for a lower MAE can allow a small number of high-variance or anomalous points to deviate further, which increases squared errors and thus leads to a higher RMSE. The improvements can be attributed to the following design choices:

- DSTFGCN combines TCN-based local temporal modeling with temporal graph convolution to capture both short-term variations and long-range temporal dependencies, which benefits multi-horizon forecasting.
- The node-independent temporal graph convolution block explicitly models heterogeneous temporal dynamics across sensors, rather than enforcing shared temporal transformations for all nodes.
- DSTFGCN integrates diffusion convolution for local topology-aware spatial aggregation with an input-dependent dynamic graph convolution block to capture global and evolving spatial relationships.

5.6. Forecasting results and visualization

To more clearly demonstrate the predictive ability of the model, we randomly selected the daily traffic of a node in three datasets for

Table 5
Ablation study of DSTFGCN on six datasets.

Dataset	Metric	DSTFGCN-A	DSTFGCN-B	DSTFGCN-C	DSTFGCN-D	DSTFGCN
PeMS03	MAE	15.22	14.87	14.83	14.84	14.34
	RMSE	26.08	25.37	24.81	23.95	24.24
	MAPE	14.88%	14.9%	15.09%	15.18%	14.60%
PeMS04	MAE	19.85	18.93	18.47	19.32	18.18
	RMSE	31.72	30.77	30.13	31.09	29.75
	MAPE	13.34%	12.62%	12.49%	12.90%	12.18%
PeMS07	MAE	21.88	21.17	19.42	21.68	19.18
	RMSE	35.72	34.85	32.21	35.35	32.33
	MAPE	9.10%	8.80%	8.36%	8.94%	8.06%
PeMS08	MAE	15.51	13.64	13.85	15.10	13.45
	RMSE	25.08	23.30	23.44	24.38	23.07
	MAPE	9.92%	8.88%	9.04%	9.52%	8.80%
METR-LA	MAE	3.88	3.64	3.88	3.77	3.19
	RMSE	7.65	7.11	7.48	7.55	6.50
	MAPE	11.01%	10.68%	10.88%	10.88%	9.30%
PEMS-BAY	MAE	2.04	1.93	1.90	1.94	1.78
	RMSE	4.72	4.33	4.44	4.47	4.00
	MAPE	4.65%	4.32%	4.50%	4.55%	3.97%

visualization. Fig. 4 shows the visualization results. Compared with the ground truth, we can observe that the predicted values are basically in line with the real values. It can be proved that DSTFGCN can accurately capture the dynamic dependency features in traffic flow, and thereby obtain accurate prediction results.

5.7. Ablation study

To demonstrate the effectiveness of the two key components in DSTFGCN, we conducted ablation studies on six datasets and designed four variants: (1) DSTFGCN w/o temporal graph blocks (DSTFGCN-A), (2) DSTFGCN w/o node-independent blocks (DSTFGCN-B), (3) DSTFGCN w/o dynamic graph blocks (DSTFGCN-C), and (4) DSTFGCN with the temporal graph convolution replaced by a temporal self-attention module (DSTFGCN-D).

- **DSTFGCN w/o temporal graph blocks (DSTFGCN-A):** We remove the entire temporal graph convolution block and model temporal dependencies solely using TCN.
- **DSTFGCN w/o node-independent blocks (DSTFGCN-B):** We remove the node-independent temporal graph convolution block, and the temporal dependencies are modeled with parameters shared across all nodes.
- **DSTFGCN w/o dynamic graph blocks (DSTFGCN-C):** We remove the dynamic graph convolution block and capture spatial correlations using only predefined adjacency matrices.
- **DSTFGCN with temporal self-attention (DSTFGCN-D):** We replace the temporal graph convolution block with a temporal self-attention module to compare graph-convolution-based temporal modeling with attention-based temporal modeling.

As shown in Table 5, the full DSTFGCN achieves the best performance across all six datasets and almost all metrics, indicating that each proposed component contributes positively to forecasting accuracy. First, DSTFGCN-A leads to the largest degradation on all datasets, showing that relying only on local temporal modeling (e.g., TCN/RNN-style dependencies) is insufficient. This confirms that capturing global temporal dependencies is crucial for traffic forecasting. Second, DSTFGCN-B consistently underperforms the full model, suggesting that node-specific temporal modeling is necessary. Compared with parameter sharing across nodes, assigning node-specific parameters enables the model to better distinguish heterogeneous traffic patterns among sensors, which translates into improved accuracy. Third, DSTFGCN-C also exhibits noticeable performance drops, implying that purely static spatial mod-

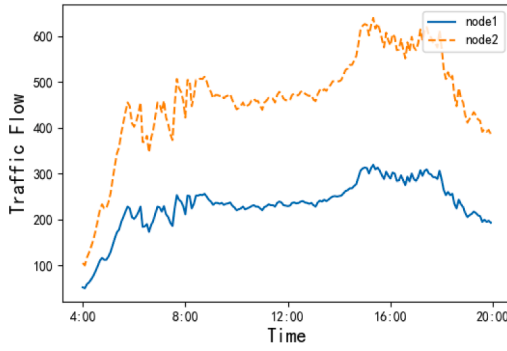
Table 6
Comparison of pooling operations.

Dataset	Metric	Plus	Max	Mean
PeMS08	MAE	13.54	13.58	13.45
	RMSE	23.15	23.16	23.07
	MAPE	8.83%	9.31%	8.80%

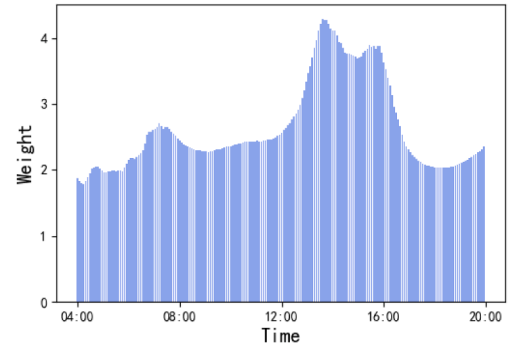
eling based on predefined adjacency matrices cannot adequately capture the evolving spatial dependencies in real traffic systems. Finally, DSTFGCN-D replaces the temporal graph convolution with a temporal self-attention module. Although attention is able to model long-range dependencies, it still performs worse than our node-independent temporal graph convolution. This result suggests that our temporal graph convolution is more suitable for traffic forecasting, because it explicitly assigns node-specific parameters and performs node-aware relation modeling in the temporal domain, whereas attention mechanisms typically rely on shared projections across nodes and may be less effective in capturing node-specific temporal dynamics. Overall, these ablation results demonstrate that combining node-independent temporal graph convolution with dynamic spatial graph convolution is critical for achieving robust performance across diverse traffic scenarios.

5.8. Visualization temporal graph

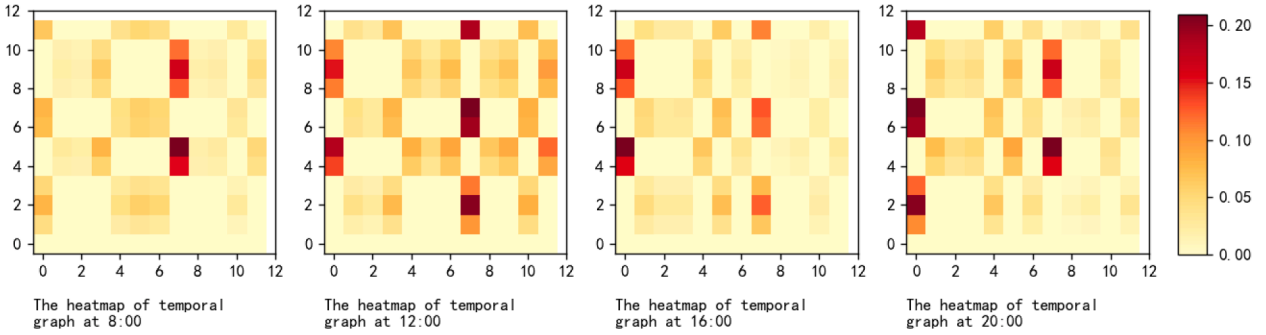
To verify the validity of the temporal graph and show more details, we visualized its corresponding graph structure. Two nodes were randomly selected for visualization. As shown in Fig. 5(a), it is the traffic flow data of two nodes (No.50 and No.61) in the PEMS08 dataset for one day. Fig. 5(c) shows a heatmap of the temporal graph for node 1 at different times. It can be observed that the correlation weights between time slots change in different periods. Fig. 5(d) shows a heatmap of the temporal graph for node 2. In comparison, it can be observed that although the flow curves of the two nodes are similar, our model learns a different traffic pattern for each node. This shows that our proposed node-independent temporal graph convolution block can learn different temporal correlation for different nodes. Therefore, our node-independent temporal graph convolution block that assigns a set of different parameters to each node performs better than the methods learning shared temporal correlation.



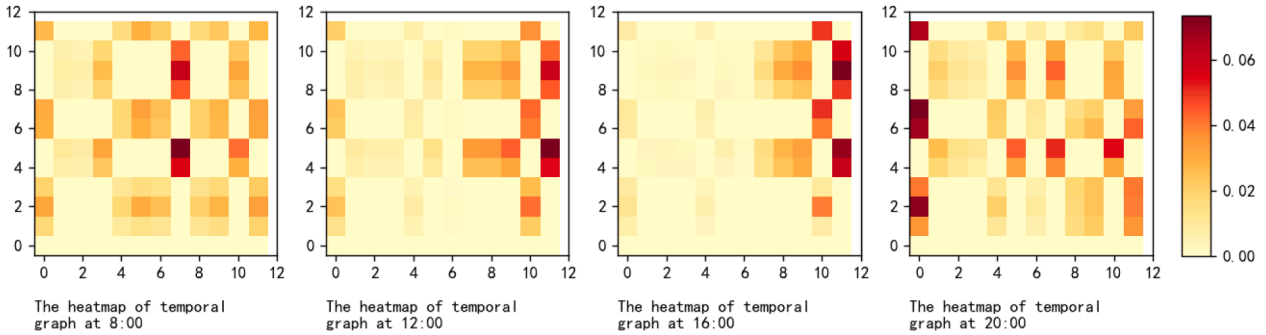
(a) The traffic flow of two nodes



(b) The weight of two nodes



(c) Heatmap of the temporal graph for node 1



(d) Heatmap of the temporal graph for node 2

Fig. 5. Visualization of temporal graph and dynamic graph.

5.9. Visualization spatial dynamic graph

The visualization results of the spatial dynamic graph are shown in Fig. 5(b). It is a statistical plot of the weights between nodes 1 and 2 in Fig. 5(a). It can be observed that the weight reaches two peaks at around 7 o'clock and 15 o'clock. This corresponds to similar fluctuations in the two time periods on the flow curve. In other time periods, the similarity of traffic flow decreases, and the corresponding weight also decreases. This shows that the dynamic graph we generate can dynamically learn the spatial correlation between different nodes as the input changes.

As shown in Table 6, we evaluate different pooling operations in the dynamic graph generation module. The results indicate that mean pooling consistently performs best among the compared choices, suggesting it provides a more reliable aggregation of temporal information for constructing dynamic connectivity patterns. Therefore, we adopt mean pooling as the default pooling strategy in our model.

Table 7

Study of spatio-temporal graph.

Dataset	Metric	Attention	Adaptive	DSTFGCN
PeMS04	MAE	19.34	19.75	18.18
	RMSE	30.97	33.99	29.75
	MAPE	13.05%	13.00%	12.18%
PeMS08	MAE	15.05	14.48	13.45
	RMSE	24.20	23.46	23.07
	MAPE	9.55%	9.17%	8.80%

5.10. Study of temporal and dynamic graph

To validate the effectiveness of the proposed temporal graph and dynamic graph, we compare DSTFGCN with two alternative graph-generation schemes. First, we replace the temporal graph with an

Table 8
Efficiency analysis of DSTFGCN.

Method	Training time (s/epoch)	Inference time (s)	Parameters (K)
LSTM	567.69	16.73	573
DCRNN	508.94	13.42	388
STGCN	28.16	7.98	426
ASTGCN	175.08	5.21	598
HSTGCN	153.19	17.07	/
GDGCN	39.60	4.07	46
DSTFGCN	36.40	3.67	80

attention-based temporal graph to examine the difference between attention mechanisms and our node-specific temporal graph convolution in capturing distinct temporal patterns across nodes. The attention-based temporal graph is computed as:

$$A_{attention} = \text{Softmax}\left(\frac{(W_q X)(W_k X)^T}{\sqrt{d}}\right) \quad (16)$$

where $X \in R^{N \times T \times d}$ is the input and $W_q, W_k \in R^{d \times d}$ are learnable parameters. Here, $A_{attention} \in R^{N \times T \times T}$ denotes the resulting attention-based graph. Second, we replace the dynamic graph with a adaptive graph defined by:

$$A_{adaptive} = \text{Softmax}(E_1 E_2^T) \quad (17)$$

where $E_1, E_2 \in R^{N \times d}$ are learnable node embeddings and $A_{adaptive} \in R^{N \times N}$. As reported in Table 7, DSTFGCN consistently achieves the best performance on both datasets, indicating that the proposed node-specific temporal graph convolution models global temporal dependencies more effectively than attention. Moreover, replacing the dynamic spatial graph with the adaptive graph leads to noticeable degradation, which supports that our dynamic graph better captures time-varying spatial correlations than a static adaptive graph. Overall, these results validate the necessity of jointly using the node-specific temporal graph and the input-dependent dynamic spatial graph in DSTFGCN.

5.11. Efficiency analysis

To evaluate computational efficiency, we report the average training time per epoch, inference time, and the number of parameters for DSTFGCN and several efficient baselines on PeMS08. As shown in Table 8, DSTFGCN achieves a favorable trade-off between efficiency and model size, with fast training/inference and a relatively small parameter budget. While STGCN trains faster, it requires substantially more parameters and delivers noticeably lower accuracy. GDGCN is more compact, but its node-shared temporal graph limits its ability to model node-specific temporal patterns, leading to inferior performance. By learning node-specific temporal transformations, DSTFGCN slightly increases the parameter count compared with GDGCN but yields clearly improved forecasting accuracy.

To evaluate the GPU memory consumption of our model, we conducted additional experiments. As shown in Table 9, “Tra-A/Tra-R” denote the peak GPU memory allocated/reserved during training, while “Val-A/Val-R” denote the corresponding peaks during validation; reserved memory is typically higher due to PyTorch’s caching allocator. Overall, the validation stage tends to consume less memory than training because it avoids gradient storage. We also observe that PeMS07 requires substantially more GPU memory, mainly because it contains many more sensors (883 nodes), whereas the other datasets have only a few hundred nodes, leading to a much larger spatial graph and higher memory footprint. Overall, these results suggest that DSTFGCN can be trained and evaluated with predictable memory patterns, and its memory demand scales reasonably with the problem size.

Table 9
Peak GPU memory (MiB) on six datasets.

Dataset	Tra-A	Tra-R	Val-A	Val-R
PeMS03	3241.60	3570.00	3143.38	3570.00
PeMS04	3550.08	3898.00	3433.67	3898.00
PeMS07	8683.14	9240.00	8430.45	9240.00
PeMS08	1216.64	1410.00	1150.53	1410.00
METR-LA	1247.92	1478.00	1212.90	1478.00
PEMS-BAY	1009.87	1310.00	982.07	1310.00

Table 10
Hyper-parameter comparison of L and d .

Setting	PeMS04			PeMS08		
	MAE	RMSE	MAPE	MAE	RMSE	MAPE
$L = 5$	18.29	30.24	12.22%	13.56	23.37	8.93%
$L = 6$	18.23	30.01	12.22%	13.45	23.07	8.80%
$L = 7$	18.38	30.07	12.28%	13.48	23.21	8.86%
$L = 8$	18.18	29.75	12.18%	13.53	23.26	9.12%
$d = 16$	18.35	30.01	12.58%	13.99	23.25	9.36%
$d = 32$	18.41	30.01	12.59%	13.56	23.12	8.84%
$d = 48$	18.23	30.04	12.23%	13.45	23.07	8.80%
$d = 64$	18.18	29.75	12.18%	13.49	23.15	8.73%

5.12. Verification of hyperparameters

We investigate two key hyperparameters: the number of ST-layers L and the hidden dimension of the input layer. Specifically, we vary $L \in \{5, 6, 7, 8\}$ and the hidden dimension $d \in \{16, 32, 48, 64\}$, and evaluate the performance on PeMS04 and PeMS08. The results are summarized in Table 10. Overall, PeMS08 achieves the best performance with $L = 6$ and $d = 48$, while PeMS04 prefers a deeper setting with $L = 8$ and a larger hidden dimension $d = 64$. These results provide practical guidance for selecting appropriate hyperparameters in our model.

6. Conclusion

We propose a novel Dynamic Spatial-Temporal Fusion Graph Convolutional Network (DSTFGCN) for traffic flow forecasting. In the temporal domain, DSTFGCN first employs a TCN to capture local temporal correlations, and then applies a node-independent temporal graph convolution block to learn node-specific global temporal dependencies. In the spatial domain, it first uses a diffusion convolution block to model local, topology-driven correlations, and then generates an input-conditioned adjacency matrix via a dynamic graph generation module to capture global spatial correlations with a dynamic graph convolution block. Finally, the temporal and spatial representations are fused and fed into a prediction module to produce the forecasting results. Extensive experiments on six public datasets demonstrate that DSTFGCN consistently outperforms recent baselines. In future work, we will further improve the computational efficiency of the model for real-world deployment by exploring lightweight designs.

CRediT authorship contribution statement

Tianyi Pan: Writing – original draft, Visualization, Software, Resources, Methodology, Investigation; **Xinyuan Zhou:** Writing – review & editing, Visualization, Investigation; **Shiyong Lan:** Writing – review & editing, Supervision, Project administration, Investigation, Funding acquisition, Conceptualization; **Wenwu Wang:** Writing – review & editing, Investigation; **Hongyu Yang:** Writing – review & editing; **Zheng Li:** Writing – review & editing; **Zhiang Hou:** Writing – review & editing; **Yao Ren:** Writing – review & editing.

Data Availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the National Natural Science Foundation of China under Grant 62371324. This work was also supported by the 2035 Innovation Pilot Program of Sichuan University, China.

References

- Abellana, D.P.M. (2021). Short-term traffic flow forecasting using the autoregressive integrated moving average model in Metro Cebu (Philippines). *International Journal of Applied Decision Sciences*, 14, 565–587. <https://api.semanticscholar.org/CorpusID:239222519>.
- Ali, A., Zhu, Y., & Zakarya, M. (2022). Exploiting dynamic spatio-temporal graph convolutional neural networks for citywide traffic flows prediction. *Neural Networks*, 145, 233–247. <https://doi.org/10.1016/j.neunet.2021.10.021>
- Bai, S., Kolter, J.Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv:1803.01271*. <https://api.semanticscholar.org/CorpusID:4747877>.
- Bu, N., Duan, Z., Dang, W., & Zhao, J. (2026). Dynamic graph transformation with multi-task learning for enhanced spatio-temporal traffic prediction. *Neural Networks*, 193, 107963. <https://doi.org/10.1016/j.neunet.2025.107963>
- Cao, C.-Y., Bao, Y.-X., Hou, Y., & Shi, Q. (2025). Traffic speed prediction network based on multi-view spatio-temporal graph convolution network. *Computers and Electrical Engineering*, 127, 110558. <https://doi.org/10.1016/j.compeleceng.2025.110558>
- Chen, A., Cao, S., & Zhang, H. (2024). Distinguish the indistinguishable: Spatial personalized transformer for traffic flow forecast. In *Web and big data: 8th international joint conference, APWeb-WAIM 2024, Jinhuu, China, August 30, - September 1, 2024, proceedings, part II* (p. 390–404). Berlin, Heidelberg: Springer-Verlag. https://doi.org/10.1007/978-981-97-7235-3_26
- Chen, C., Petty, K.F., Skabardonis, A., Varaiya, P.P., & Jia, Z. (2001). Freeway performance measurement system: Mining loop detector data. *Transportation Research Record*, 1748, 96–102. <https://api.semanticscholar.org/CorpusID:108891582>.
- Chen, Y., Huo, J., Lin, F., & Yan, H. (2025). Beyond homophily in spatial-temporal traffic flow forecasting. *Neural Networks*, 183, 106950. <https://doi.org/10.1016/j.neunet.2024.106950>
- Chen, Y., Qin, Y., Li, K., Yeo, C.K., & Li, K. (2023a). Adaptive spatial-temporal graph convolution networks for collaborative local-global learning in traffic prediction. *IEEE Transactions on Vehicular Technology*, 72(10), 12653–12663. <https://doi.org/10.1109/TVT.2023.3276752>
- Chen, Z., Ma, M., Li, T., Wang, H., & Li, C. (2023b). Long sequence time-series forecasting with deep learning: A survey. *Information Fusion*, 97, 101819. <https://api.semanticscholar.org/CorpusID:258412923>.
- Chi, H., Lu, Y., Zhu, Y., Ke, W., & Mao, H. (2025). Dynamic spatiotemporal graph convolutional network collaborative pre-training learning for traffic flow prediction. *Knowledge-Based Systems*, 329, 114339. <https://doi.org/10.1016/j.knsys.2025.114339>
- Dong, P., & Zhang, X. (2025). ST-GTrans: Spatio-temporal graph transformer with road network semantic awareness for traffic flow prediction. *Neural Networks*, 190, 107623. <https://doi.org/10.1016/j.neunet.2025.107623>
- Fan, J., Weng, W., Chen, Q., Wu, H., & Wu, J. (2025). PDG2Seq: Periodic dynamic graph to sequence model for traffic flow prediction. *Neural Networks*, 183, 106941. <https://doi.org/10.1016/j.neunet.2024.106941>
- Guo, K., Tian, D., Hu, Y., Sun, Y., Qian, Z.S., Zhou, J., Gao, J., & Yin, B. (2024). Contrastive optimized graph convolution network for traffic forecasting. *Neurocomputing*, 602, 128249. <https://doi.org/10.1016/j.neucom.2024.128249>
- Guo, S., Lin, Y., Feng, N., Song, C., & Wan, H. (2019). Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. In *AAAI conference on artificial intelligence*. <https://api.semanticscholar.org/CorpusID:58565947>.
- Guo, S., Lin, Y., Wan, H., Li, X., & Cong, G. (2021). Learning dynamics and heterogeneity of spatial-temporal graph data for traffic forecasting. *IEEE Transactions on Knowledge and Data Engineering*, 34, 5415–5428. <https://api.semanticscholar.org/CorpusID:234018156>.
- Hamilton, W.L., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Neural information processing systems*. <https://api.semanticscholar.org/CorpusID:4755450>.
- He, K., Chen, X., Wu, Q., Yu, S., & Zhou, Z. (2022). Graph attention spatial-temporal network with collaborative global-local learning for citywide mobile traffic prediction. *IEEE Transactions on Mobile Computing*, 21(4), 1244–1256. <https://doi.org/10.1109/TMC.2020.3020582>
- Huang, X., Zhang, B., Feng, S., Ye, Y., & Li, X. (2023). Interpretable local flow attention for multi-step traffic flow prediction. *Neural Networks*, 161, 25–38. <https://doi.org/10.1016/j.neunet.2023.01.023>
- Huber, P.J. (1964). Robust estimation of a location parameter. *Annals of Mathematical Statistics*, 35, 492–518. <https://api.semanticscholar.org/CorpusID:121252793>.
- Jiang, J., Han, C., Zhao, W.X., & Wang, J. (2023). PDFormer: Propagation delay-aware dynamic long-range transformer for traffic flow prediction. *arXiv:2301.07945*. <https://api.semanticscholar.org/CorpusID:255999991>.
- Lan, S., Ma, Y., Huang, W., Wang, W., Yang, H., & Li, P. (2022). DSTAGNN: Dynamic spatial-temporal aware graph neural network for traffic flow forecasting. In *International conference on machine learning*. <https://api.semanticscholar.org/CorpusID:250340665>.
- Li, F., Feng, J., Yan, H., Jin, G., Jin, D., & Li, Y. (2021). Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution. *ACM Transactions on Knowledge Discovery from Data*, 17, 1–21. <https://api.semanticscholar.org/CorpusID:233476480>.
- Li, Q., Yang, X., Wang, Y., Wu, Y., & He, D. (2022). Spatial-temporal traffic modeling with a fusion graph reconstructed by tensor decomposition. *IEEE Transactions on Intelligent Transportation Systems*, 25, 1749–1760. <https://api.semanticscholar.org/CorpusID:254563934>.
- Li, Y., Yu, R., Shahabi, C., & Liu, Y. (2017). Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. *arXiv: Learning*. <https://api.semanticscholar.org/CorpusID:3508727>.
- Lin, Z., Feng, J., Lu, Z., Li, Y., & Jin, D. (2019). DeepSTN+: Context-aware spatial-temporal neural network for crowd flow prediction in metropolis. In *AAAI conference on artificial intelligence*. <https://api.semanticscholar.org/CorpusID:70300566>.
- Liu, H., Dong, Z., Jiang, R., Deng, J., Deng, J., Chen, Q., & Song, X. (2023). Spatio-temporal adaptive embedding makes vanilla transformer SOTA for traffic forecasting. In *Proceedings of the 32nd ACM international conference on information and knowledge management*. <https://api.semanticscholar.org/CorpusID:264350711>.
- Liu, Y., Wu, X., Tang, Y., Li, X., Sun, D., & Zheng, L. (2024). Decomposition with feature attention and graph convolution network for traffic forecasting. *Knowledge-Based Systems*, 300, 112193. <https://api.semanticscholar.org/CorpusID:270858140>.
- Lou, J., Zhang, X., Wang, R., Liu, Z., Zhao, K., & Shen, Q. (2024). HSFE: A hierarchical spatial-temporal feature enhanced framework for traffic flow forecasting. *Information Sciences*, 679, 121070. <https://www.sciencedirect.com/science/article/pii/S002002524009848>. <https://doi.org/10.1016/j.ins.2024.121070>
- Luo, Y., Zheng, J., Wang, X., Tao, Y., & Jiang, X. (2024). GT-LSTM: A spatio-temporal ensemble network for traffic flow prediction. *Neural Networks*, 171, 251–262. <https://doi.org/10.1016/j.neunet.2023.12.016>
- Luo, Y., Zheng, J., Wang, X., Wenjuan, E., Jiang, X., & Zhu, Z. (2025). ST-GMLP: A concise spatial-temporal framework based on gated multi-layer perceptron for traffic flow forecasting. *Neural Networks*, 184, 107074. <https://doi.org/10.1016/j.neunet.2024.107074>
- Ma, Y., Lou, H., Yan, M., Sun, F., & Li, G. (2023). Spatio-temporal fusion graph convolutional network for traffic flow forecasting. *Information Fusion*, 104, 102196. <https://api.semanticscholar.org/CorpusID:266322703>.
- Seo, Y., Defferrard, M., Vandergheynst, P., & Bresson, X. (2016). Structured sequence modeling with graph convolutional recurrent networks. In *International conference on neural information processing*. <https://api.semanticscholar.org/CorpusID:2687749>.
- Song, C., Lin, Y., Guo, S., & Wan, H. (2020). Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting. In *AAAI conference on artificial intelligence*. <https://api.semanticscholar.org/CorpusID:214274349>.
- Sun, B., Cheng, W., Goswami, P., & Bai, G. (2017). Flow-aware WPT k-nearest neighbours regression for short-term traffic prediction. *2017 IEEE Symposium on Computers and Communications (ISCC)*, (pp. 48–53). <https://api.semanticscholar.org/CorpusID:31233865>.
- Ta, X., Liu, Z., Hu, X., Yu, L., Sun, L., & Du, B. (2022). Adaptive spatio-temporal graph neural network for traffic forecasting. *Knowledge-Based Systems*, 242, 108199. <https://api.semanticscholar.org/CorpusID:246323064>.
- Tucker, L.R. (1966). Some mathematical notes on three-mode factor analysis. *Psychometrika*, 31, 279–311. <https://api.semanticscholar.org/CorpusID:44301099>.
- Vlahogianni, E.I., Golias, J.C., & Karlaftis, M.G. (2004). Short-term traffic forecasting: Overview of objectives and methods. *Transport Reviews*, 24, 533–557. <https://api.semanticscholar.org/CorpusID:111285053>.
- Wang, L., Hong, S., Chi, H., Xie, C., Zhu, Y., & Mao, H. (2025). Hybrid spatio-temporal graph neural network with attention fusion for traffic flow prediction. *Knowledge-Based Systems*, 324, 113813. <https://doi.org/10.1016/j.knsys.2025.113813>
- Weng, W., Fan, J., Wu, H., Hu, Y., Tian, H., Zhu, F., & Wu, J. (2023). A decomposition dynamic graph convolutional recurrent network for traffic forecasting. *Pattern Recognition*, 142, 109670. <https://doi.org/10.1016/j.patcog.2023.109670>
- Weng, W., Jiang, H., Wu, M., Han, X., Gao, H., Shen, G., & Kong, X. (2025). Let's group: A plug-and-play subgraph learning method for memory-efficient spatio-temporal graph modeling. In *Proceedings of the thirty-fourth international joint conference on artificial intelligence IJCAI '25*. <https://doi.org/10.24963/ijcai.2025/386>
- Weng, W., Wu, M., Jiang, H., Kong, W., Kong, X., & Xia, F. (2025). Pattern-matching dynamic memory network for dual-mode traffic prediction. *IEEE Transactions on Intelligent Transportation Systems*, 26(9), 13603–13617. <https://doi.org/10.1109/TITS.2025.3564564>
- Wu, C.-H., Ho, J.-M., & Lee, D.T. (2004). Travel-time prediction with support vector regression. *IEEE Transactions on Intelligent Transportation Systems*, 5(4), 276–281. <https://doi.org/10.1109/TITS.2004.837813>
- Wu, Z., Pan, S., Long, G., Jiang, J., & Zhang, C. (2019). Graph wavenet for deep spatial-temporal graph modeling. In *International joint conference on artificial intelligence*. <https://api.semanticscholar.org/CorpusID:173990443>.

- Xu, Y., Han, L., Zhu, T., Sun, L., Du, B., & Lv, W. (2023). Generic dynamic graph convolutional network for traffic flow forecasting. *Information Fusion*, 100, 101946. <https://api.semanticscholar.org/CorpusID:260301511>.
- Yan, H., Ma, X., & Pu, Z. (2021). Learning dynamic and hierarchical traffic spatiotemporal features with transformer. *IEEE Transactions on Intelligent Transportation Systems*, 23, 22386–22399. <https://api.semanticscholar.org/CorpusID:233209832>.
- Yao, H., Wu, F., Ke, J., Tang, X., Jia, Y., Lu, S., Gong, P., Ye, J., & Li, Z.J. (2018). Deep multi-view spatial-temporal network for taxi demand prediction. In *AAAI conference on artificial intelligence*. <https://api.semanticscholar.org/CorpusID:3600371>.
- Yu, B., Lee, Y., & Sohn, K. (2020). Forecasting road traffic speeds by considering area-wide spatio-temporal dependencies based on a graph convolutional neural network (GCN). *Transportation Research Part C-emerging Technologies*, 114, 189–204. <https://api.semanticscholar.org/CorpusID:214037476>.
- Yu, H., Wu, Z., Wang, S., Wang, Y., & Ma, X. (2017). Spatiotemporal recurrent convolutional networks for traffic prediction in transportation networks. *Sensors (Basel, Switzerland)*, 17. <https://api.semanticscholar.org/CorpusID:3506872>.
- Zhang, D., Wang, P., Ding, L., Wang, X., & He, J. (2025a). Spatio-temporal contrastive learning-based adaptive graph augmentation for traffic flow prediction. *IEEE Transactions on Intelligent Transportation Systems*, 26(1), 1304–1318. <https://doi.org/10.1109/ITITS.2024.3487982>
- Zhang, Q., Gao, X., Wang, H., Yiu, S.M., & Yin, H. (2025b). Efficient traffic prediction through spatio-temporal distillation. In *AAAI conference on artificial intelligence*. <https://doi.org/10.1609/aaai.v39i1.32096>.
- zhang, X., Chen, X., Tang, H., Wu, Y., Shen, H., & Li, J. (2024). AdpSTGCN: Adaptive spatial-temporal graph convolutional network for traffic forecasting. *Knowledge-Based Systems*, 301, 112295. <https://doi.org/10.1016/j.knosys.2024.112295>
- Zheng, C., Fan, X., Wang, C., & Qi, J. (2019). GMAN: A graph multi-attention network for traffic prediction. <https://arxiv.org/abs/1911.08415>.